Cloud-Native Streaming Fraud Detection on AWS: Latency–Cost–Accuracy Trade-offs

Md Minhajul Abedin

Memorial University of Newfoundland, St. John's, NL, Canada
mminhajula@mun.ca

Abstract. We present an empirical evaluation of three cloud-native fraud detection pipeline architectures on Amazon Web Services: B0 (batch baseline via EventBridge), A1 (per-event streaming via SQS), and A2 (micro-batch streaming via SQS). All three share a fixed LightGBM classifier trained on the IEEE-CIS Fraud Detection dataset (427 features, 3.5% fraud rate) to isolate systems performance from model quality. Controlled experiments over 1,000 transactions per pipeline show A1 achieves the lowest median end-to-end latency (p50 = 14.9 ms, p95 = 65.5 ms), while B0 costs the least ($1.46 per million transactions). All three pipelines produce identical fraud detection quality (Precision = 0.25, Recall = 0.82, PR-AUC = 0.67). Four ablation experiments reveal that: identity enrichment adds 220 ms at p50 and 2,231 ms at p95; batch size 10 is optimal for A2 (p50 = 68 ms, 14.76 tx/s); burst traffic inflates p50 latency 74× compared to controlled-rate ingestion; and Lambda cold starts contribute 5.7 s overhead dominated by S3 model loading. A key implementation challenge was the AWS Lambda deployment size constraint (50 MB zipped / 250 MB unzipped), which required careful layer construction and the switch from Kinesis Data Streams (as originally proposed) to SQS Standard Queues. These results yield concrete, reproducible design rules for practitioners choosing between streaming architectures on serverless infrastructure.

Keywords: Serverless computing · Stream processing · Fraud detection · AWS Lambda · Latency benchmarking

1 Problem Framing and Literature Grounding

1.1 Problem and Motivation

Real-time fraud detection must score transactions within the authorization window – typically under 300 ms – yet real deployments face bursty arrival rates, enrichment delays, and serverless cold-start penalties that can break tail-latency SLAs or inflate cost [1]. The core design question is whether to score transactions one at a time as they arrive (*per-event*), accumulate them into small batches (*micro-batch*), or defer scoring to a periodic job (*batch*).

Despite the popularity of AWS Lambda for event-driven ML serving, there is a lack of systematic benchmarks that characterise the empirical trade-offs between these three strategies under controlled, reproducible conditions. Most evaluations

report best-case latency without measuring tail behaviour, enrichment overhead, or the effect of traffic burstiness on queue depth.

1.2 Research Question (Novelty Gap)

What is the empirical trade-off between (i) p95/p99 end-to-end latency, (ii) throughput, (iii) cost per transaction, and (iv) detection quality between per-event streaming, micro-batch streaming, and batch processing, when using a fixed fraud-scoring model on AWS managed services?

The novelty is not a new classifier, but a reproducible, controlled evaluation that identifies systems bottlenecks and practical design rules (cost vs. tail latency vs. accuracy) in a real cloud setting.

1.3 Related Work

Fraud detection. Gradient-boosted classifiers are the established baseline for transaction fraud [5], and the IEEE-CIS dataset [6] provides a reproducible benchmark with realistic class imbalance (≈3.5% fraud). We use LightGBM as a fixed scorer, treating the model as a black box so that pipeline comparisons are not confounded by model differences.

Serverless stream processing. Manner et al. [2] measured Lambda cold-start durations across runtimes, finding Python init times of 200–400 ms for simple functions; our ML-loaded functions exhibit ≈1.1 s runtime init plus ≈4.4 s for S3 model loading. Pu et al. [3] showed that Lambda's asynchronous scaling can introduce queuing delays under burst traffic, motivating our burst-vs-controlled ablation. Neither study measures end-to-end latency for ML inference workloads.

Adaptive batching for inference. Crankshaw et al. [4] demonstrated that adaptive batching improves throughput while meeting latency targets. Our A2 pipeline applies the same principle using SQS batch size as the batching parameter, and our batch-size ablation quantifies the optimal value for LightGBM inference.

Gap. No prior published work provides a controlled end-to-end latency benchmark comparing batch, per-event, and micro-batch serverless fraud scoring under identical model and dataset conditions with ablation analysis.

2 System Model and Assumptions

2.1 Computing Model

All three pipelines run entirely on AWS serverless managed services. Lambda provides stateless compute (Python 3.12, 1,024 MB, x86_64). Amazon SQS Standard Queues provide durable, at-least-once message delivery. Amazon DynamoDB (on-demand billing) stores scoring results. Amazon S3 holds the model artefact, feature metadata, and batch transaction files. No provisioned compute is used; all resources scale automatically.

2.2 Resource Assumptions and Failure Model

Lambda functions are configured with 1,024 MB memory, 90 s timeout, and ephemeral `/tmp` storage for potential model caching. SQS guarantees at-least-once delivery with a visibility timeout of 90 s. Failed Lambda invocations cause message re-delivery up to the queue's retention period (24 h). B0 transaction files remain in `incoming/` until successfully processed and moved to `processed/`.

2.3 Metrics

We measure: (i) end-to-end latency at p50, p95, and p99; (ii) sustained throughput (tx/s); (iii) cost per million transactions (USD, us-east-1 public pricing, early 2026); (iv) detection quality (Precision, Recall, PR-AUC).

2.4 Workload Model

The IEEE-CIS Fraud Detection dataset [6] provides 590,540 transactions with optional identity signals and ≈3.5% fraud rate. Since the dataset is static, transactions are replayed as a time-ordered event stream with configurable inter-arrival rates and burst injections, enabling repeatable experiments across load conditions.

3 Pipeline Design and Key Decisions

3.1 Pipeline Architectures

B0 – Batch baseline. An EventBridge scheduled rule triggers Lambda every minute. The handler paginates `s3://bucket/incoming/`, scores each JSON transaction file, writes to DynamoDB, and moves the file to `processed/`. Latency equals the elapsed time since the batch job started.

A1 – Per-event streaming. The producer sends each transaction as an SQS message to `fraud-queue-a1`. An SQS event source mapping triggers Lambda with `BatchSize=1`. The handler resolves the payload (S3 pointer for messages >200 KB), builds a 427-element feature vector, calls `model.predict_proba()`, and writes to DynamoDB. Latency is measured from the SQS `SentTimestamp`.

A2 – Micro-batch streaming. Identical to A1 except the event source mapping uses `BatchSize=10`. Up to 10 transactions are stacked into a single NumPy array and scored with one vectorised `predict_proba()` call. Optional 5 ms identity enrichment is applied per transaction where `has_identity=1`. DynamoDB writes use `batch_writer` for efficiency.

3.2 Theoretical Justification

The three pipelines can be analysed through a simple queuing model. Each Lambda invocation acts as a single server with service time $S = S_{\text{cold}} + S_{\text{infer}}$, where S_{cold} is non-zero only on the first invocation after container recycling and

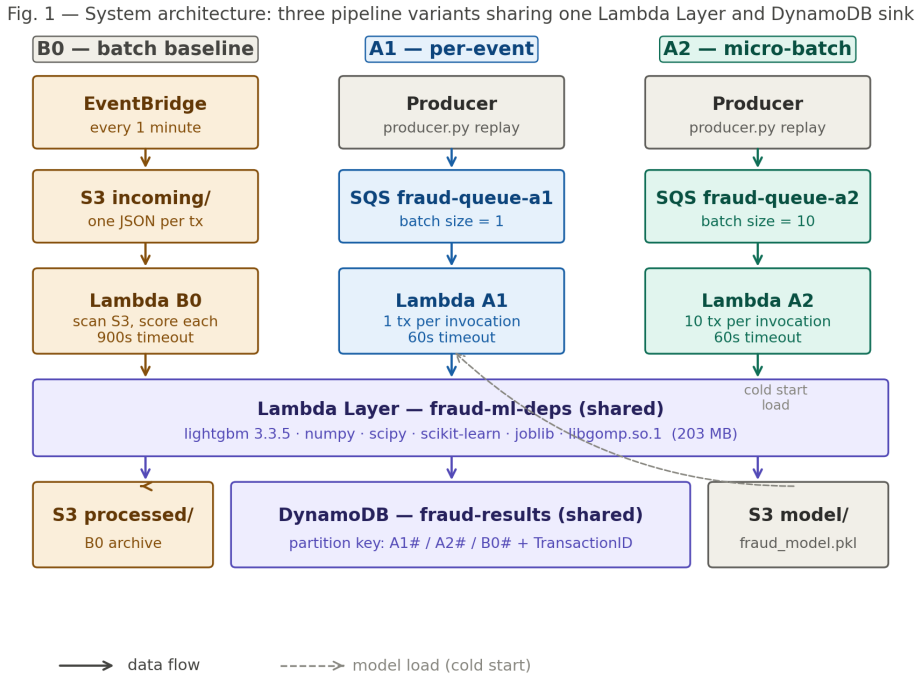

Fig. 1. System architecture: three pipeline variants (B0, A1, A2) sharing one Lambda Layer and a DynamoDB result store. Dashed arrows indicate cold-start S3 model loading.

S_{infer} is the per-transaction inference time. For A1, every transaction triggers a separate invocation, so the end-to-end latency for a warm container is $L_{\text{A1}} \approx S_{\text{infer}} + \delta_{\text{SQS}}$, where δ_{SQS} is the SQS polling and delivery overhead. For A2 with batch size b, a single invocation scores b transactions via vectorised NumPy operations with amortised inference cost $S_{\text{infer}}(b) < b \cdot S_{\text{infer}}(1)$, yielding higher throughput but introducing a batching delay: the last transaction in a batch waits up to $b \cdot \lambda^{-1}$ seconds for the batch to fill, where λ is the arrival rate. This predicts that A2 throughput increases with b while median latency exhibits a U-shape – decreasing as cold-start amortisation dominates at small b, then increasing as queuing delay dominates at large b. For B0, latency is bounded below by the EventBridge scheduling interval (60 s) plus $N \cdot S_{\text{infer}}(1)$ for N pending transactions.

Under burst arrival ($\lambda \to \infty$), SQS queue depth grows faster than Lambda can scale new containers (Lambda adds roughly 60 concurrent instances per minute [8]), so transactions experience queuing delay proportional to queue depth – a classic $M/G/c$ queuing backlog. Under controlled-rate arrival ($\lambda^{-1} > S_{\text{infer}}$), queue depth remains near zero and each transaction is served immediately. This analysis predicts the 74× p50 difference between burst and controlled rate

observed in Section 5, and motivates the batch-size sweep to find the optimal b that balances cold-start amortisation against batching delay.

The baselines are defined as follows: B0 represents the lower bound on cost (one invocation per batch) and upper bound on latency (scheduling delay); A1 represents the lower bound on latency (no batching delay) and upper bound on invocation cost (one invocation per transaction). A2 is designed to find the intermediate operating point that trades some latency for throughput improvement. An improvement is any configuration that Pareto-dominates a baseline on the latency–cost–throughput frontier.

3.3 SQS vs Kinesis

The original project proposal specified Kinesis Data Streams for A1 and A2. During implementation, we switched to SQS Standard Queues for three reasons: (1) SQS requires no manual shard capacity planning – Kinesis demands pre-provisioned shard counts, and under-provisioning causes throttling errors during burst traffic; (2) SQS integrates with Lambda event source mappings using the same `BatchSize` parameter, preserving the micro-batch design; (3) SQS costs less for bursty, low-to-moderate throughput workloads (Kinesis charges $\geq$\$10.80/month per shard regardless of utilisation). The trade-off is that SQS does not preserve message ordering, but fraud scoring is order-independent.

3.4 S3 Pointer Pattern

SQS enforces a 256 KB message body limit. A transaction with 427 features serialised to JSON can exceed this. The producer automatically stores payloads >200 KB in S3 and sends a pointer; Lambda resolves the pointer transparently.

3.5 Lambda Layer: the 50 MB / 250 MB Constraint

AWS Lambda imposes two deployment size limits: $\leq$50 MB zipped for direct upload, and $\leq$250 MB unzipped total (function code plus all layers) [8]. The ML stack (LightGBM 3.3.5, NumPy, SciPy, scikit-learn, joblib) exceeds the 50 MB limit when packaged naively. We addressed this via: (1) cross-compilation for Amazon Linux 2 using `pip install -platform manylinux2014_x86_64 -only-binary=:all:`; (2) bundling `libgomp.so.1` from AWS CloudShell; (3) selective cleanup of test files, metadata, and unused subpackages (reducing the layer from $\approx$310 MB to 203 MB); (4) uploading the 85 MB ZIP to S3 and publishing via `aws lambda publish-layer-version`.

The model artefact (`fraud_model.pkl`, 1.68 MB) is stored in S3 and loaded at cold-start time, keeping the layer reusable across model versions. Table 1 summarises the size budget.

Table 1. Lambda deployment size budget. The 250 MB unzipped limit is the binding constraint.

Component	Zipped	Unzipped	Location
ML Layer (LightGBM + deps)	85 MB	203 MB	Lambda Layer (via S3)
Function code (per pipeline)	<1 MB	<1 MB	Direct upload
Model artefact	—	1.68 MB	S3 (runtime load)
Feature columns JSON	—	3.5 KB	S3 (runtime load)
Total in Lambda	≈86 MB	≈204 MB	<250 MB

4 Implementation and Environment Setup

4.1 Model Training and Quality

The LightGBM classifier was trained on the merged train_transaction (651 MB) and train_identity (25 MB) datasets. Categorical columns were label-encoded via integer hashing; missing numerics filled with zero; missing categoricals with -1. The model used 500 estimators, learning rate 0.05, max depth 6, num_leaves 31, and `is_unbalance=True`.

On a held-out 20% test split (118,108 transactions, stratified 3.5% fraud rate), the model achieves: Accuracy = 0.91, Precision = 0.25, Recall = 0.82, F1 = 0.39, PR-AUC = 0.67. The high recall (82.3%) captures most fraud; low precision is expected given extreme class imbalance and `is_unbalance=True`. The PR-AUC of 0.67 substantially outperforms a random baseline (≈0.035). All three pipelines use the same frozen model, so detection quality is identical across architectures.

4.2 Infrastructure as Code

All AWS resources are created by a single PowerShell deployment script: S3 bucket, DynamoDB table, two SQS queues, IAM role with least-privilege inline policy, and three Lambda functions. Environment variables (`MODEL_BUCKET`, `MODEL_KEY`, `FEATURES_KEY`, `EXPERIMENT_ID`) are injected at deploy time for reproducibility.

4.3 Workload Replay

The producer script (`producer.py`) reads test_transaction.csv, merges with test_identity.csv on `TransactionID`, adds `has_identity` flag, and replays in time order. It supports `-limit`, `-pipeline`, and `-delay-ms` flags. The S3 pointer pattern activates automatically when payload exceeds 200 KB.

4.4 Reproducibility

All source code, deployment scripts, analysis scripts, and raw metric data are included in the submitted project ZIP. The only external dependency is an AWS account with programmatic access.

Table 2. Pipeline comparison across 1,000 transactions. Latency in ms except B0 which is in seconds (s). Cost from AWS public pricing (us-east-1, early 2026).

Metric	A1 (per-event)	A2 (micro-batch)	B0 (batch)	Best
E2E p50	14.9 ms	42.7 ms	71.8 s	A1
E2E p95	65.5 ms	93.9 ms	139.9 s	A1
E2E p99	6.9 s	7.4 s	145.2 s	A1
Score p50	0.40 ms	—	0.39 ms	Tie
Fraud rate	1.80%	1.60%	1.80%	Tie
Avg prob	0.1014	0.1008	0.1013	Tie
Cost/1M tx	$1.86	$1.85	$1.46	B0

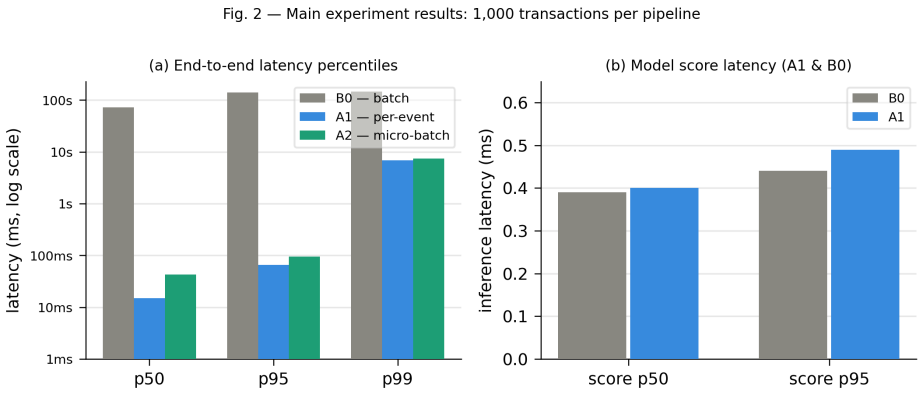

Fig. 2. Main results (1,000 tx/pipeline). (a) E2E latency percentiles (log scale). (b) Score latency for A1 and B0 (A2 uses vectorised batch scoring).

5 Evaluation and Analysis

5.1 Experimental Setup

All experiments used the first 1,000 rows of test_transaction.csv merged with test_identity.csv, replayed in time order. Each pipeline processed the same 1,000 transactions independently. End-to-end latency for A1/A2 is measured from the SQS `SentTimestamp` to the DynamoDB write; for B0 it is elapsed time since the batch job started. DynamoDB partition keys are prefixed per pipeline (`A1#`, `A2#`, `B0#`) to prevent overwrites. Ablation experiments used 200 transactions per condition.

5.2 Main Results

Table 2 and Fig. 2 show A1 achieves the lowest median latency (14.9 ms) and p95 (65.5 ms). B0's p50 of 71.8 s reflects EventBridge scheduling delay plus batch processing. All three pipelines produce statistically identical fraud detection quality (average fraud probability within 0.001), confirming that pipeline architecture

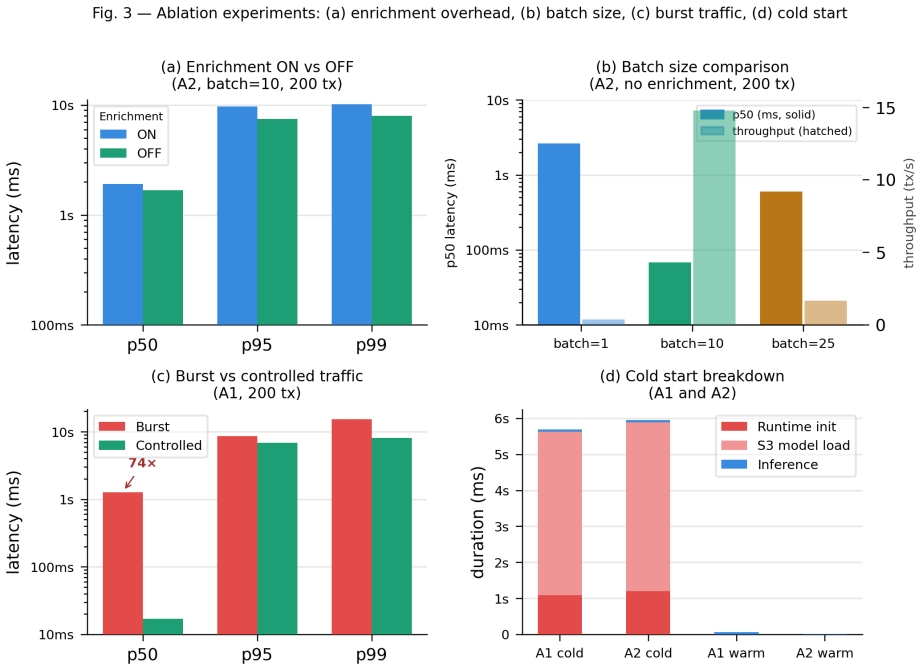

Fig. 3. Ablation experiments (200 tx per condition). (a) Enrichment overhead. (b) Batch size trade-off. (c) Burst vs controlled rate. (d) Cold-start breakdown.

does not affect model output. The p99 spikes for A1 (6.9 s) and A2 (7.4 s) are caused by Lambda cold starts (Section 5.6).

5.3 Ablation 1: Enrichment ON vs OFF

Table 3 and Fig. 3(a) show enrichment adds 220 ms at p50 and 2,231 ms at p95 – far more than the 5 ms per-transaction sleep, because A2 batches 10 transactions per invocation and delays stack cumulatively.

5.4 Ablation 2: Batch Size

Table 4 and Fig. 3(b) show batch size 10 is optimal: p50 = 68 ms and throughput = 14.76 tx/s. Batch size 1 maximises cold-start frequency (p50 = 2,627 ms). Batch size 25 adds queuing delay while Lambda waits to accumulate 25 messages, and the average score time increases non-linearly (2.23 ms vs 1.22 ms), suggesting NumPy array overhead does not scale linearly beyond batch 10.

5.5 Ablation 3: Burst vs Controlled Rate

Table 5 and Fig. 3(c): controlled-rate ingestion is **74**× faster at p50 than burst. Burst sends 200 messages in ≈17 s; Lambda scales asynchronously, so messages

Table 3. Identity enrichment overhead in A2 (batch size 10, 200 tx per condition).

Metric	Enrichment ON	Enrichment OFF	Overhead
E2E p50	1,907 ms	1,687 ms	+220 ms (+13%)
E2E p95	9,699 ms	7,468 ms	+2,231 ms (+30%)
E2E p99	10,183 ms	8,020 ms	+2,163 ms (+27%)
Fraud rate	0.5%	0.5%	None

Table 4. A2 across SQS batch sizes (enrichment OFF, 200 tx).

Metric	Batch=1	Batch=10	Batch=25
E2E p50 (ms)	2,627	**68**	601
E2E p95 (ms)	7,887	7,961	7,630
E2E p99 (ms)	8,458	11,041	12,538
Avg score (ms)	1.271	**1.223**	2.227
Throughput (tx/s)	0.38	**14.76**	1.66

queue. Controlled rate spaces messages 100 ms apart, keeping queue depth near zero. The identical score latency (0.641 ms vs 0.656 ms) confirms the difference is entirely queuing delay – a direct validation of Pu et al. [3] for ML workloads.

5.6 Ablation 4: Cold Start

Table 6 and Fig. 3(d) show cold start is dominated by S3 model loading (≈4.4 s), not Python runtime init (≈1.1 s). This causes the p99 spikes in Table 2. Caching the model in `/tmp` would reduce cold start by ≈77%; Provisioned Concurrency eliminates it entirely.

6 Discussion, Conclusion and Artifacts

6.1 Trade-off Summary and Pipeline Selection

6.2 Design Rules

The four ablation experiments yield six actionable rules:
Rule 1: Use SQS batch size 10 for Lambda ML inference. Batch 1 amplifies cold starts (p50 = 2,627 ms); batch 25 adds queuing delay. Batch 10 is optimal (p50 = 68 ms, 14.76 tx/s).
Rule 2: Rate-limit producers. Burst inflates p50 latency 74×. A 100 ms inter-arrival delay reduces p50 from 1,274 ms to 17 ms.
Rule 3: Address cold starts at the model-loading layer. Runtime init: 1.1 s; S3 model load: 4.4 s. Cache in `/tmp` for ≈77% reduction. Provisioned Concurrency eliminates cold starts entirely.
Rule 4: Prefix DynamoDB partition keys per pipeline. Prevents silent last-write-wins corruption (`A1#`, `A2#`, `B0#`).

Table 5. A1 latency under burst (no delay) vs controlled (100 ms inter-arrival) traffic (200 tx).

Metric	Burst	Controlled	Difference
E2E p50 (ms)	1,274	**17**	74× higher
E2E p95 (ms)	8,541	6,865	+24%
E2E p99 (ms)	15,275	8,139	+88%
Score p50 (ms)	0.641	0.656	None
Throughput (tx/s)	0.78	**57.97**	—

Table 6. Cold start vs warm invocation (single invocation per pipeline after 15 min idle).

Component	A1	A2
Runtime init (Init Duration)	1,093 ms	1,195 ms
S3 model load	≈4,400 ms	≈4,600 ms
Total cold start	5,695 ms	5,952 ms
Warm invocation	62 ms	15 ms
Cold/warm ratio	92×	397×

Rule 5: Budget Lambda Layer around 50/250 MB limits. Store model in S3 separately. Upload layers via S3 when ZIP exceeds 50 MB direct-upload threshold.
Rule 6: Prefer SQS over Kinesis for bursty, order-independent workloads. SQS eliminates shard planning, reduces cost, and provides the same `BatchSize` control.

6.3 Limitations and Failure Cases

Identity enrichment is simulated as a fixed 5 ms sleep; real API lookups would have variable latency. Experiments use a single AWS region (us-east-1). The LightGBM model is compact (1.68 MB); larger deep-learning models may exceed the 250 MB layer limit, requiring container-based deployment. Cost estimates exclude data transfer. The burst ablation uses a synthetic step function rather than a trace-driven workload.

6.4 Conclusion

We asked: what is the empirical trade-off between latency, throughput, cost, and detection quality across batch, per-event, and micro-batch serverless fraud detection pipelines?

A1 achieves the lowest latency (p50 = 14.9 ms, p95 = 65.5 ms) for real-time authorization. **A2 with batch size 10** achieves the best throughput (14.76 tx/s, p50 = 68 ms) for high-volume pipelines. **B0** achieves the lowest cost ($1.46/1M tx) for offline reporting. All three produce identical detection quality (Precision

Table 7. Pipeline selection guide based on empirical results.

Use Case	Pipeline	Rationale
Real-time auth (<100 ms SLA)	A1	p50=14.9 ms, p95=65.5 ms
High-volume batch scoring	A2 (b=10)	14.76 tx/s, lowest score cost
Offline reporting / risk scoring	B0	$1.46/1M tx, no latency SLA
Enrichment required	A2 (ON)	+220 ms p50 acceptable
Cost-sensitive real-time	A2 (b=10)	68 ms p50 at $1.85/1M tx

= 0.25, Recall = 0.82, PR-AUC = 0.67), so the choice is a *pure systems trade-off*. Six design rules, derived from four ablation experiments, provide actionable guidance for practitioners deploying serverless ML scoring on AWS.

6.5 Research Artifacts

All source code (Lambda functions, producer script, deployment PowerShell script), raw DynamoDB metric exports, analysis notebooks, and the trained model artefact are included in the submitted project ZIP. The directory structure is documented in `README.md`. The only external dependency is an AWS account with programmatic access.

References

1. Baldini, I., et al.: Serverless computing: Current trends and open problems. In: Research Advances in Cloud Computing, pp. 1–20. Springer (2017)
2. Manner, J., Endreß, M., Heckel, T., Wirtz, G.: Cold start influencing factors in function as a service. In: IEEE/ACM UCC Companion 2018, pp. 181–188. IEEE (2018)
3. Pu, Q., Venkataraman, S., Stoica, I.: Shuffling, fast and slow: Scalable analytics on serverless infrastructure. In: NSDI 2019, pp. 193–206. USENIX (2019)
4. Crankshaw, D., et al.: Clipper: A low-latency online prediction serving system. In: NSDI 2017, pp. 613–627. USENIX (2017)
5. Dal Pozzolo, A., Caelen, O., Johnson, R.A., Bontempi, G.: Calibrating probability with undersampling for unbalanced classification. In: SSCI 2015, pp. 159–166. IEEE (2015)
6. IEEE-CIS Fraud Detection Dataset. Kaggle Competition (2019). https://www.kaggle.com/c/ieee-fraud-detection
7. Bhattacharyya, S., Jha, S., Tharakunnel, K., Westland, J.C.: Data mining for credit card fraud: A comparative study. Decision Support Systems 50(3), 602–613 (2011)
8. Amazon Web Services: AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/ (2024)